MINI PROJECT REPORT

ELECTRICITY DEMAND AND RETAIL SALES FORECASTING USING LINEAR REGRESSION

PROJECT AREA	Machine Learning / Time-Series Forecasting
ALGORITHM	Linear Regression
TOOLS	Python, Pandas, NumPy, Matplotlib, Scikit-learn
DATA	Synthetic daily demand and monthly sales observations

AIM

To build and evaluate a machine learning model that forecasts electricity demand and monthly retail sales from time-based observations. The project demonstrates data preparation, chronological train-test splitting, regression, evaluation metrics, visualization, and future prediction.

OBJECTIVES

- Generate and inspect time-series datasets.
- Preprocess dates and construct useful time-based features.
- Train a Linear Regression model using historical observations.
- Evaluate predictions using MSE, RMSE, and R^2 score.
- Visualize historical trends and compare actual values with predictions.
- Produce a 12-month retail sales forecast.

Note: the datasets in this report are synthetic and intended for academic demonstration; results should not be treated as operational forecasts.

EXPERIMENT 1 — CODE (1/5)

Electricity Demand Forecasting using Linear Regression

```
# MINI PROJECT
# Electricity Demand Forecasting using Linear Regression

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Reproducible synthetic dataset
np.random.seed(42)

dates = pd.date_range(
    start="2024-01-01",
    periods=500,
    freq="D"
)

days = np.arange(500)
trend = 180 + 0.12 * days
weekly = 9 * np.sin(2 * np.pi * days / 7)
seasonal = 18 * np.sin(2 * np.pi * days / 120)
noise = np.random.normal(0, 5, 500)
demand = trend + weekly + seasonal + noise
```

The dataset is generated with a gradual demand trend, weekly variation, longer seasonal variation, temperature values, and random noise. A fixed seed makes the demonstration reproducible.

EXPERIMENT 1 — CODE (2/5)

Electricity Demand Forecasting using Linear Regression

```
temperature = (
    25
    + 7 * np.sin(2 * np.pi * days / 180)
    + np.random.normal(0, 1.5, 500)
)

df = pd.DataFrame({
    "Date": dates,
    "Temperature": temperature,
    "Demand_kWh": demand
})

print("Original Dataset:")
print(df.head())
print("\nDataset Shape:")
print(df.shape)
print("\nMissing Values:")
print(df.isnull().sum())

# Preprocess data
df["Date"] = pd.to_datetime(df["Date"])
df = df.sort_values("Date")
df["DayOfWeek"] = df["Date"].dt.dayofweek
df["Days"] = (df["Date"] - df["Date"].min()).dt.days
```

EXPERIMENT 1 — CODE (3/5)

Electricity Demand Forecasting using Linear Regression

```
# Create a lag feature from previous demand
df["Previous_Demand"] = df["Demand_kWh"].shift(1)
df = df.dropna().reset_index(drop=True)

print("\nPreprocessed Dataset:")
print(df.head())

# Select input features and target
X = df[[
    "Days",
    "Temperature",
    "DayOfWeek",
    "Previous_Demand"
]]
y = df["Demand_kWh"]

# Preserve chronological order for time series
split_index = int(len(df) * 0.8)
X_train = X.iloc[:split_index]
X_test = X.iloc[split_index:]
y_train = y.iloc[:split_index]
y_test = y.iloc[split_index:]

print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))
```

The first 80% of observations are used for training and the final 20% for testing. Chronological splitting helps avoid training on future observations when evaluating a time-series model.

EXPERIMENT 1 — CODE (4/5)

Electricity Demand Forecasting using Linear Regression

```
# Build and train the model
model = LinearRegression()
model.fit(X_train, y_train)
print("Linear Regression model trained successfully.")

# Predict on unseen test observations
y_pred = model.predict(X_test)

# Evaluate model performance
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\nMODEL EVALUATION")
print("MSE:", round(mse, 4))
print("RMSE:", round(rmse, 4))
print("R² Score:", round(r2, 4))

comparison = pd.DataFrame({
    "Date": df.iloc[split_index:]["Date"].values,
    "Actual Demand": y_test.values,
    "Predicted Demand": y_pred
})
print(comparison.head(10).to_string(index=False))
```

EXPERIMENT 1 — CODE (5/5)

Electricity Demand Forecasting using Linear Regression

```
# Compare actual and predicted demand
plt.figure(figsize=(12, 6))
plt.plot(
    comparison["Date"],
    comparison["Actual Demand"],
    label="Actual Demand"
)
plt.plot(
    comparison["Date"],
    comparison["Predicted Demand"],
    label="Predicted Demand"
)
plt.xlabel("Date")
plt.ylabel("Demand (kWh)")
plt.title("Actual vs Predicted Electricity Demand")
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# Visualize complete demand history
plt.figure(figsize=(12, 6))
plt.plot(df["Date"], df["Demand_kWh"])
plt.xlabel("Date")
plt.ylabel("Demand (kWh)")
plt.title("Historical Electricity Demand")
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

print("Experiment completed successfully.")
```

EXPERIMENT 1 — OUTPUT

```
Original Dataset:
      Date  Temperature  Demand_kWh
2024-01-02    28.108421  187.407209
2024-01-03    23.390444  194.134306
2024-01-04    26.576153  194.695923
2024-01-05    24.998248  179.146690
2024-01-06    25.484849  175.313707

Dataset Shape: (500, 3)
Missing Values:
Date           0
Temperature    0
Demand_kWh     0

Preprocessed Dataset:
      Date  Temperature  Demand_kWh  DayOfWeek  Previous_Demand  Days
2024-01-02    28.108421  187.407209          1       182.483571     0
2024-01-03    23.390444  194.134306          2       187.407209     1
2024-01-04    26.576153  194.695923          3       194.134306     2
2024-01-05    24.998248  179.146690          4       194.695923     3

Training Samples: 399
Testing Samples: 100
Linear Regression model trained successfully.
```

EXPERIMENT 1 — MODEL EVALUATION

```
MODEL EVALUATION
Mean Squared Error (MSE): 48.6624
Root Mean Squared Error (RMSE): 6.9758
R² Score: 0.7624

Actual vs Predicted Demand (first 10 test rows):
      Date  Actual Demand  Predicted Demand
2025-02-04         242.65            249.80
2025-02-05         248.99            242.05
2025-02-06         246.73            245.34
2025-02-07         238.68            241.76
2025-02-08         230.83            233.53
2025-02-09         237.41            225.47
2025-02-10         235.43            241.58
2025-02-11         246.49            238.24
2025-02-12         248.92            245.30
2025-02-13         245.36            245.47
```

MSE measures the average squared prediction error. RMSE expresses the error in the target's units, while R^2 describes the proportion of target variance explained by the model on the test set.

EXPERIMENT 1 — VISUALIZATION

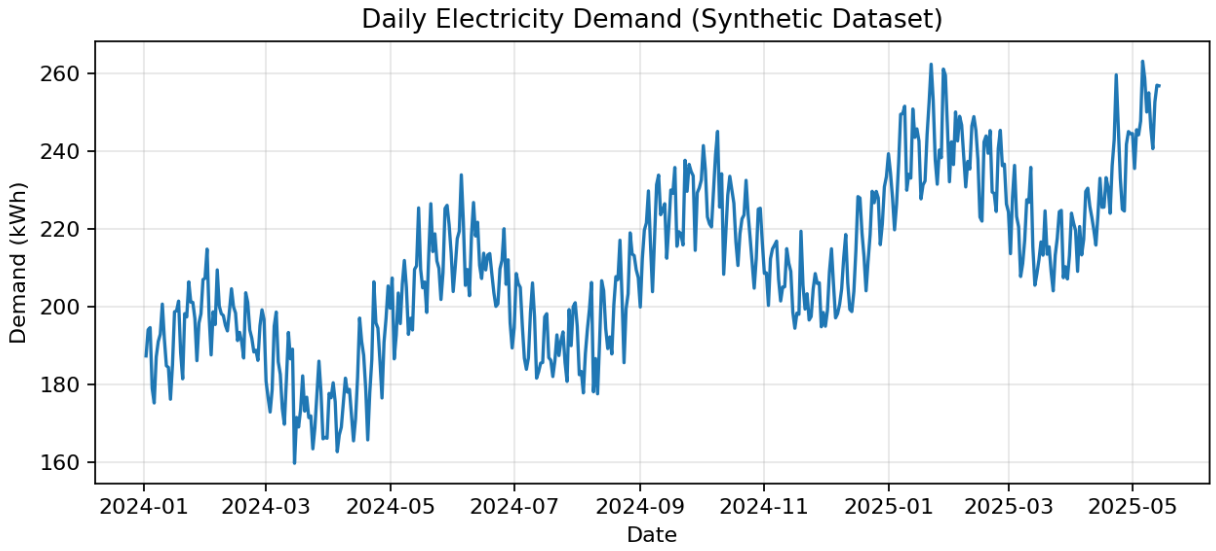


Figure 1. Historical daily electricity demand. The synthetic series combines an overall upward trend with recurring short- and medium-term fluctuations.

EXPERIMENT 1 — ACTUAL VS PREDICTED

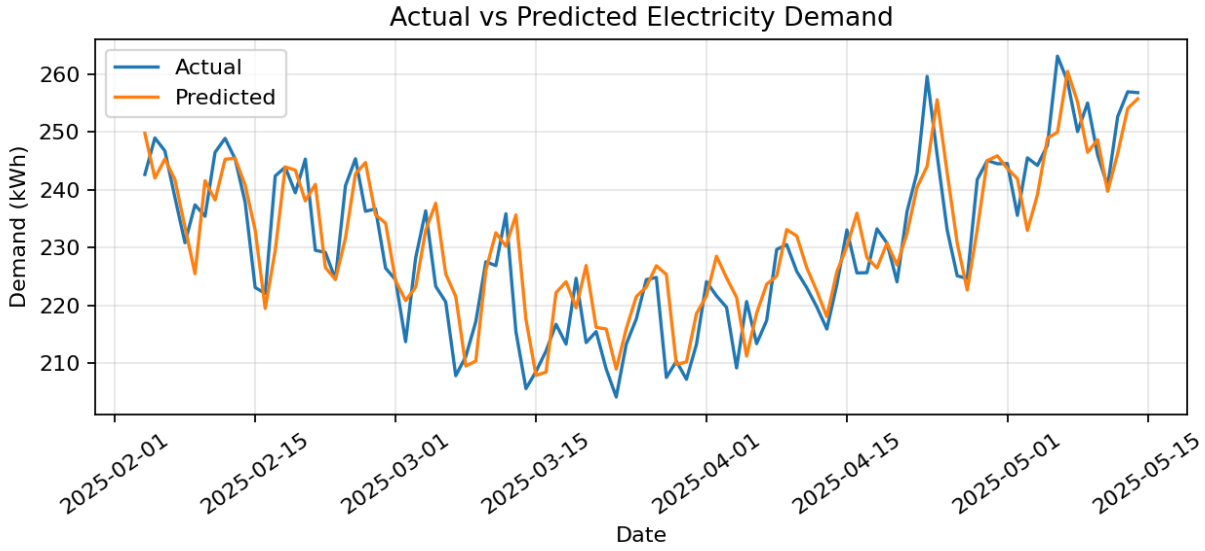


Figure 2. Comparison of measured synthetic demand and model predictions on the held-out test period. The plot supports visual inspection of the prediction fit and deviations.

EXPERIMENT 2 — CODE (1/4)

Monthly Retail Sales Forecasting using Linear Regression

```
# EXPERIMENT 2
# Monthly Retail Sales Forecasting using Linear Regression

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

np.random.seed(42)

# Generate monthly retail sales observations
dates = pd.date_range(
    start="2014-01-01",
    periods=144,
    freq="MS"
)
t = np.arange(len(dates))
trend = 420 + 2.6 * t
seasonality = 75 * np.sin(2 * np.pi * t / 12 - 0.8)
noise = np.random.normal(0, 18, len(dates))
sales = trend + seasonality + noise

df = pd.DataFrame({
    "Date": dates,
    "Sales_Units": np.round(sales, 0).astype(int)
})
```

Monthly sales are generated from a gradual growth trend, a 12-month seasonal component, and random variation. This makes the example useful for illustrating the limitations of a simple linear trend model.

EXPERIMENT 2 — CODE (2/4)

Monthly Retail Sales Forecasting using Linear Regression

```
print("Retail Sales Dataset:")
print(df.head())
print("\nDataset Shape:")
print(df.shape)
print("\nMissing Values:")
print(df.isnull().sum())

# Prepare chronological features
df["Date"] = pd.to_datetime(df["Date"])
df = df.sort_values("Date")
df["Month_Number"] = np.arange(len(df))
df["Year"] = df["Date"].dt.year
df["Month"] = df["Date"].dt.month

print("\nPreprocessed Dataset:")
print(df.head())

# Plot sales history
plt.figure(figsize=(12, 6))
plt.plot(df["Date"], df["Sales_Units"])
plt.xlabel("Date")
plt.ylabel("Units Sold")
plt.title("Monthly Retail Sales")
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

EXPERIMENT 2 — CODE (3/4)

Monthly Retail Sales Forecasting using Linear Regression

```
# Define feature and target
X = df[["Month_Number"]]
y = df["Sales_Units"]

# Chronological 80:20 split
split_index = int(len(df) * 0.8)
X_train = X.iloc[:split_index]
X_test = X.iloc[split_index:]
y_train = y.iloc[:split_index]
y_test = y.iloc[split_index:]

print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))

# Train model and predict test values
model = LinearRegression()
model.fit(X_train, y_train)
print("Linear Regression model trained successfully.")
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
```

The model uses month index as its input feature. Because this simple model does not explicitly encode month-of-year seasonality, it may not capture seasonal peaks and troughs.

EXPERIMENT 2 — CODE (4/4)

Monthly Retail Sales Forecasting using Linear Regression

```
print("\nMODEL EVALUATION")
print("Mean Squared Error (MSE):", round(mse, 4))
print("Root Mean Squared Error (RMSE):", round(rmse, 4))
print("R² Score:", round(r2, 4))

comparison = pd.DataFrame({
    "Date": df.iloc[split_index:]["Date"].values,
    "Actual Sales": y_test.values,
    "Predicted Sales": y_pred
})
print("\nActual vs Predicted Values:")
print(comparison.head(15).to_string(index=False))

# Plot test-set comparison
plt.figure(figsize=(12, 6))
plt.plot(comparison["Date"], comparison["Actual Sales"], label="Actual")
plt.plot(comparison["Date"], comparison["Predicted Sales"], label="Predicted")
plt.xlabel("Date")
plt.ylabel("Units Sold")
plt.title("Actual vs Predicted Retail Sales")
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

EXPERIMENT 2 — DATASET OUTPUT

```
Retail Sales Dataset:
      Date  Sales_Units  Month_Number  Year  Month
2014-01-01          391             0  2014      1
2014-02-01          419             1  2014      2
2014-03-01          445             2  2014      3
2014-04-01          468             3  2014      4
2014-05-01          515             4  2014      5

Dataset Shape: (144, 5)
Missing Values:
Date          0
Sales_Units   0

Preprocessed Dataset:
      Date  Sales_Units  Month_Number  Year  Month
2014-01-01          391             0  2014      1
2014-02-01          419             1  2014      2
2014-03-01          445             2  2014      3
2014-04-01          468             3  2014      4
2014-05-01          515             4  2014      5

Training Samples: 115
Testing Samples: 29
Linear Regression model trained successfully.
```

EXPERIMENT 2 — MODEL EVALUATION

```
MODEL EVALUATION
Mean Squared Error (MSE): 2658.3523
Root Mean Squared Error (RMSE): 51.5592
R² Score: 0.1404

Actual vs Predicted Sales (first 15 test rows):
      Date  Actual Sales  Predicted Sales
2023-08-01           746           721.96
2023-09-01           747           724.53
2023-10-01           671           727.10
2023-11-01           658           729.67
2023-12-01           676           732.24
2024-01-01           698           734.82
2024-02-01           735           737.39
2024-03-01           767           739.96
2024-04-01           771           742.53
2024-05-01           844           745.10
2024-06-01           797           747.67
2024-07-01           807           750.25
2024-08-01           757           752.82
2024-09-01           733           755.39
2024-10-01           709           757.96
```

EXPERIMENT 2 — FUTURE FORECAST

```
Future 12-Month Sales Forecast:
      Date  Predicted Sales
2026-01-01           796.54
2026-02-01           799.11
2026-03-01           801.68
2026-04-01           804.25
2026-05-01           806.82
2026-06-01           809.39
2026-07-01           811.97
2026-08-01           814.54
2026-09-01           817.11
2026-10-01           819.68
2026-11-01           822.25
2026-12-01           824.82
```

The forecast extends the learned linear trend into the next 12 months. It is an illustrative baseline: seasonal effects and real-world business drivers would need to be modelled for a production-quality sales forecast.

EXPERIMENT 2 — HISTORICAL SALES

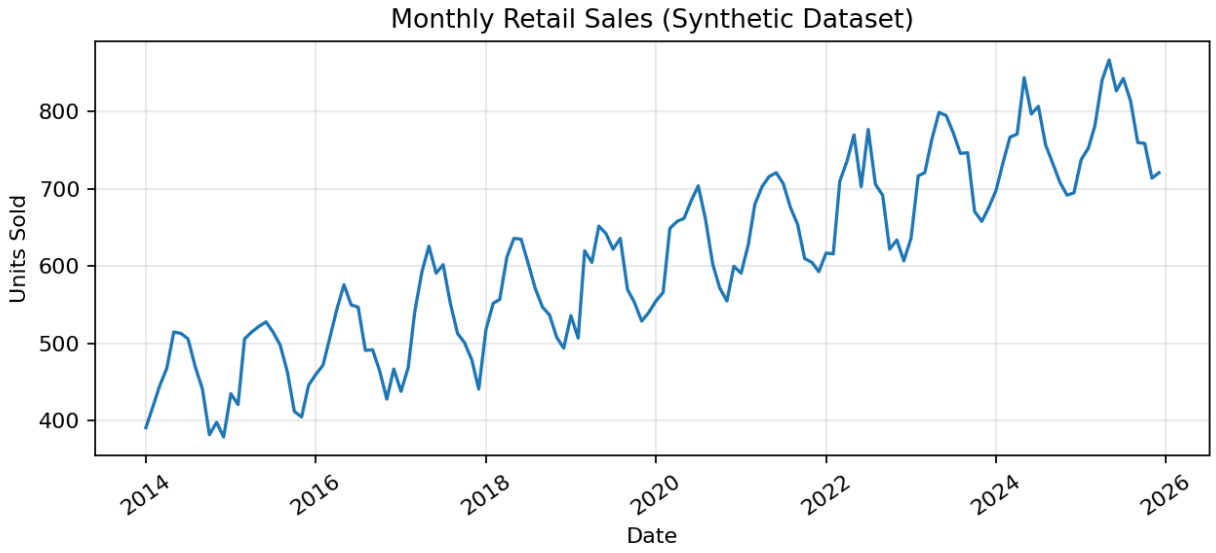


Figure 3. Monthly retail sales over the 12-year synthetic history. The pattern shows growth over time alongside recurring seasonal movement.

EXPERIMENT 2 — ACTUAL VS PREDICTED SALES

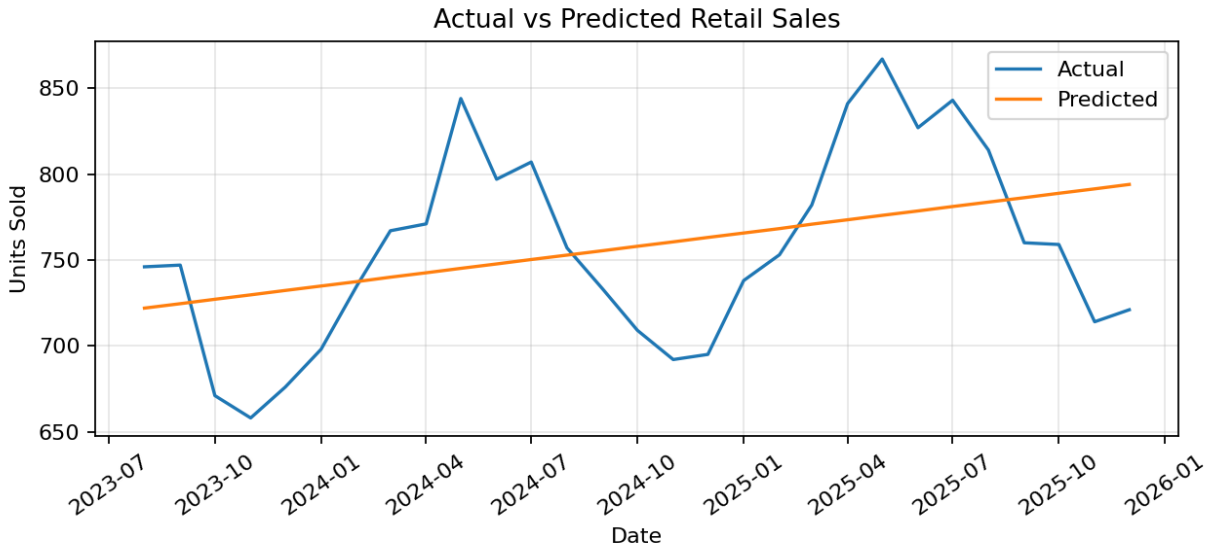


Figure 4. Test-set actual values compared with predictions from the linear trend model. The difference between seasonal observations and a smooth trend illustrates why additional features or time-series methods may improve performance.

CONCLUSION AND FUTURE ENHANCEMENTS

Conclusion

This mini project demonstrated an end-to-end machine learning workflow for two forecasting tasks: daily electricity demand and monthly retail sales. The data were inspected and preprocessed, time-based features were created, Linear Regression models were trained with a chronological 80:20 split, and predictions were evaluated using MSE, RMSE, and R^2. Visualizations helped compare historical patterns with model estimates.

Key Observations

- Chronological splitting is appropriate for time-dependent observations.
- Lag and calendar features can provide useful context for demand forecasting.
- A simple linear trend is easy to interpret but may not represent strong seasonal patterns.
- Evaluation metrics should be interpreted alongside plots and the scale of the target variable.

Future Enhancements

1. Use real electricity or retail datasets from trusted sources.
2. Add holiday indicators, weather forecasts, promotions, and calendar effects.
3. Compare Linear Regression with Random Forest, Gradient Boosting, and ARIMA/Prophet-style models.
4. Apply time-series cross-validation and monitor performance as new data arrives.
5. Package the model in a simple dashboard or API for demonstration.

Software Stack: Python • NumPy • Pandas • Matplotlib • Scikit-learn